Neural Machine Translation from English to Urdu Using Vanilla RNN Encoder–Decoder Architecture

Muhammad Idrees[1]

Department of Computer Science,
National University of Computer and Emerging Sciences (FAST-NUCES),
Islamabad, Pakistan
i230582@nu.edu.pk

Abstract. This paper presents the design, implementation, and comprehensive evaluation of a Neural Machine Translation (NMT) system for English-to-Urdu translation using a vanilla Recurrent Neural Network (RNN) encoder–decoder architecture. The system deliberately avoids the use of LSTM, GRU, or Transformer components, allowing a focused study of the simplest recurrent architecture for sequence-to-sequence learning. We train on a parallel corpus of 9,103 sentence pairs (8,996 after cleaning) sourced from Kaggle, split into 80/10/10 train/validation/test partitions with seed 42. Our methodology encompasses a complete machine translation pipeline: data preprocessing with language-specific normalization, word-level tokenization (English vocabulary: 3,969 tokens; Urdu vocabulary: 4,313 tokens), vanilla RNN encoder–decoder with teacher forcing, and systematic hyperparameter optimization via Grid Search across six dimensions. Translation quality is evaluated using BLEU-1 through BLEU-4 scores with both greedy decoding and beam search (k=5). The best tuned model achieves a validation loss of 5.6000, with greedy decoding BLEU-1 of 0.1158 and beam search BLEU-4 of 0.0062 on the test set. Qualitative error analysis of 15 translated sentences reveals that 100% of outputs are classified as POOR, with dominant failure patterns including repetitive token generation and word order/missing words errors—clearly demonstrating the well-documented limitations of vanilla RNNs including vanishing gradients, context bottleneck, and lack of gating mechanisms.

Keywords: Neural Machine Translation · Vanilla RNN · Encoder–Decoder · English–Urdu · Beam Search · BLEU Score · Sequence-to-Sequence

1 Introduction

Machine Translation (MT) is the task of automatically converting text from a source language to a target language, a fundamental challenge in natural language processing (NLP). Neural Machine Translation (NMT) has revolutionized this field by replacing traditional statistical approaches with end-to-end deep

learning models [1]. The Sequence-to-Sequence (Seq2Seq) architecture, consisting of an encoder and a decoder, forms the foundation of modern NMT systems.

English-to-Urdu translation poses unique challenges due to fundamental structural differences between the two languages:

- **Word Order**: English follows Subject-Verb-Object (SVO) word order, while Urdu follows Subject-Object-Verb (SOV) order, requiring significant reordering during translation.
- **Morphological Richness**: Urdu is a morphologically rich language with complex verb conjugations, postpositions, and gender agreement, making vocabulary modeling challenging.
- **Script Differences**: Urdu uses a modified Perso-Arabic script written right-to-left, whereas English uses the Latin alphabet, creating unique preprocessing requirements.
- **Resource Scarcity**: English-Urdu is a low-resource language pair compared to English-French or English-German, limiting the available parallel data.

In this work, we deliberately constrain the architecture to use **only vanilla RNN layers** (Elman RNN [2])—without LSTM, GRU, or Transformer components—to study the fundamental capabilities and limitations of the simplest recurrent architecture for NMT. This pedagogical constraint allows us to clearly observe phenomena such as vanishing gradients, context bottleneck limitations, and the challenges of encoding long-range dependencies.

1.1 Objectives

1. Preprocess and clean the English-Urdu parallel corpus with language-specific normalization pipelines.
2. Construct word-level vocabularies with special tokens (`PAD`, `SOS`, `EOS`, `UNK`) and minimum frequency thresholds.
3. Implement a vanilla RNN encoder–decoder architecture with teacher forcing and dropout regularization.
4. Conduct systematic hyperparameter tuning via Grid Search across six hyperparameter dimensions.
5. Evaluate translation quality using BLEU-1/2/3/4 scores with both greedy and beam search decoding.
6. Perform qualitative error analysis on 15 translated sentences, identifying common failure patterns.

1.2 Organization

Section 2 describes the dataset, preprocessing pipeline, model architecture, and training setup. Section 3 presents quantitative evaluation results including hyperparameter comparison and BLEU scores. Section 4 provides qualitative error analysis and discusses limitations of the vanilla RNN approach. Section 5 summarizes findings and proposes future research directions.

2 Methodology

2.1 Dataset (Task 1)

We use the English-Urdu parallel corpus from Kaggle [8], containing 9,103 sentence pairs. The dataset is provided as an Excel spreadsheet (`english_to_urdu_dataset.xlsx`) with two columns: `eng` (English) and `urdu` (Urdu). The dataset contains Biblical text translated between English and Urdu.

Table 1: Dataset statistics before and after preprocessing.

Metric	**Value**
Original samples	9,103
Removed (invalid/duplicates/out-of-range)	107
Remaining after cleaning	8,996

2.2 Data Preprocessing (Task 1)

The preprocessing pipeline applies language-specific normalization to both English and Urdu text:

English Text Cleaning.

1. Unicode normalization using NFKD decomposition to handle accented characters and special symbols.
2. Case folding to lowercase for vocabulary reduction.
3. Removal of non-alphanumeric characters except basic punctuation (periods, commas, exclamation marks, question marks, apostrophes, hyphens).
4. Whitespace normalization (collapsing multiple spaces to single space).

Urdu Text Cleaning.

1. Unicode normalization using NFKC composition, which is more appropriate for Arabic-script languages as it preserves canonical character forms.
2. Removal of any Latin characters that may have been erroneously included.
3. Retention of all Urdu/Arabic Unicode characters, digits, and native punctuation.
4. Whitespace normalization.

Quality Filtering.

- Removal of empty entries (either source or target is empty after cleaning).
- Removal of exact duplicate sentence pairs.

– Filtering sentences outside the 1–50 word range to remove extremely short or long sequences.

Table 2 summarizes the sentence length statistics after cleaning.

Table 2: Sentence length statistics (word count) after preprocessing.

Statistic	**English**	**Urdu**
Count	8,996	8,996
Mean	20.35	22.82
Std	9.36	10.16
Min	1	1
25%	14	16
50%	20	23
75%	27	30
Max	50	50

Sample cleaned sentence pairs from the dataset:

– **EN:** *the book of the generation of jesus christ the son of david the son of abraham*
– **UR:** >

2.3 Train-Validation-Test Split (Task 2)

The cleaned dataset is split reproducibly using a fixed random seed (`seed=42`) into training, validation, and test partitions with an 80/10/10 ratio:

Table 3: Dataset split statistics.

Split	**Samples**	**Percentage**
Train	7,196	80.0%
Validation	899	10.0%
Test	901	10.0%
Total	**8,996**	**100%**

No overlap exists between any pair of splits, verified programmatically by computing set intersections of sentence pair signatures:

– Train–Val overlap: 0
– Train–Test overlap: 0
– Val–Test overlap: 0

2.4 Tokenization & Vocabulary Construction (Task 3)

We implement word-level tokenization for both languages using whitespace splitting. Vocabularies are built exclusively from training data to prevent data leakage, with a minimum frequency threshold of 2 to filter noise. Four special tokens are introduced:

Table 4: Special tokens and their indices.

Token	Index	Purpose
`<PAD>`	0	Padding for uniform batch sizes
`<SOS>`	1	Start-of-sequence marker
`<EOS>`	2	End-of-sequence marker
`<UNK>`	3	Unknown/out-of-vocabulary words

Table 5: Vocabulary statistics built from training data.

Metric	English	Urdu
Vocabulary size (after filtering)	3,969	4,313
Unique words (before filtering)	6,437	7,363
Minimum frequency threshold	2	2

The most frequent English words include: *the* (8,753), *and* (8,380), *of* (4,739), *that* (2,960), *to* (2,812). The most frequent Urdu tokens include: > (7,567), > (6,756), > (4,865), > (4,726), > (4,600).

2.5 Sequence Encoding & Batching (Task 4)

Tokenized sentences are converted to integer sequences and prepared for batch processing:

- **Encoder input**: Source tokens padded to maximum batch length using `<PAD>`.
- **Decoder input**: $\langle \text{SOS} \rangle, w_1, w_2, \ldots, w_N$ (right-shifted target, excluding last token).
- **Decoder target**: $w_1, w_2, \ldots, w_N, \langle \text{EOS} \rangle$ (left-shifted target, excluding first token).
- **Padding masks**: Binary masks indicating non-padded positions, used to ignore padded tokens in loss computation.

Batches are sorted by source sentence length in descending order to enable efficient `pack_padded_sequence` processing. Default batch size is 64.

A sample batch has the following structure:

- `encoder_input`: shape $[64, 48]$
- `decoder_input`: shape $[64, 50]$
- `decoder_target`: shape $[64, 50]$

2.6 Model Architecture (Task 5)

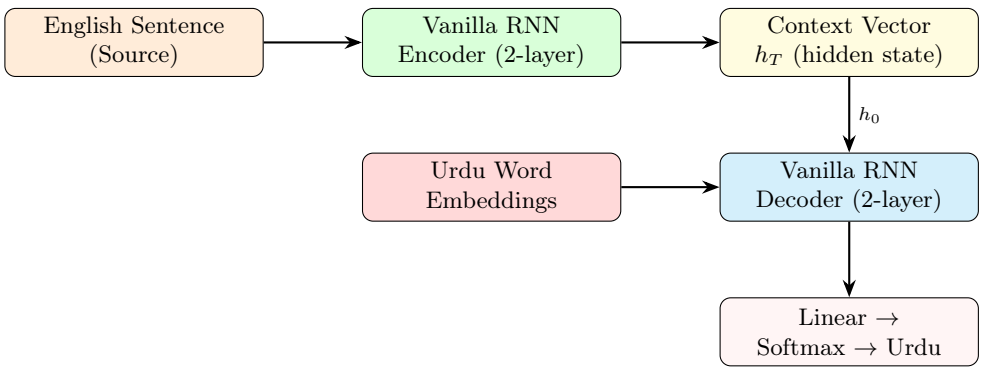


Fig. 1: Vanilla RNN encoder–decoder architecture. Strictly uses `nn.RNN` only—no LSTM, GRU, or attention mechanism.

The model architecture is verified programmatically to contain **no LSTM, GRU, or Transformer modules**. The complete model contains **6,171,865 trainable parameters**.

Encoder. The encoder is a multi-layer vanilla RNN (`nn.RNN`, Elman RNN with tanh nonlinearity) that processes the embedded source sequence:

$$\mathbf{e}_t = \text{Dropout}(\text{Embed}(x_t)) \in \mathbb{R}^{256} \tag{1}$$

$$\mathbf{h}_t = \tanh(\mathbf{W}_{ih}\mathbf{e}_t + \mathbf{b}_{ih} + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_{hh}) \in \mathbb{R}^{512} \tag{2}$$

The encoder uses `pack_padded_sequence` for efficient processing. The final hidden state $\mathbf{h}_T$ serves as the context vector for the decoder.

```
class VanillaRNNEncoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_layers,
        dropout):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx
            =0)
```

```
        self.rnn = nn.RNN(embed_dim, hidden_dim, num_layers=n_layers,
                          batch_first=True, dropout=dropout, nonlinearity=
                              'tanh')
        self.dropout = nn.Dropout(dropout)
```

Listing 1.1: Encoder architecture (vanilla RNN).

Decoder. The decoder is a matching vanilla RNN initialized with the encoder's context vector:

$$\mathbf{e}'_t = \text{Dropout}(\text{Embed}(y_{t-1})) \tag{3}$$

$$\mathbf{s}_t = \tanh(\mathbf{W}'_{ih}\mathbf{e}'_t + \mathbf{W}'_{hh}\mathbf{s}_{t-1}) \tag{4}$$

$$P(y_t|y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_o\mathbf{s}_t + \mathbf{b}_o) \tag{5}$$

The decoder generates target tokens sequentially, with each output feeding back as input for the next timestep during inference.

Teacher Forcing. During training, teacher forcing is employed with probability that decays over epochs:

$$p_{\text{tf}}^{(e)} = \max(0.2,\ 0.5 \times 0.95^e) \tag{6}$$

where e is the epoch number. This gradual reduction encourages the model to rely on its own predictions as training progresses, from an initial ratio of 0.50 down to a minimum of 0.20.

Table 6: Model architecture summary.

Component	**Specification**
Encoder Embedding	$3{,}969 \times 256$
Encoder RNN	`nn.RNN`, 2 layers, hidden=512, tanh
Decoder Embedding	$4{,}313 \times 256$
Decoder RNN	`nn.RNN`, 2 layers, hidden=512, tanh
Output Projection	Linear $512 \rightarrow 4{,}313$
Dropout	0.3
Total Parameters	6,171,865

2.7 Training Setup (Task 6)

Table 7: Training configuration.

Parameter	Value
Device	CUDA (GPU)
Optimizer	Adam (β_1=0.9, β_2=0.999)
Learning Rate	1×10^{-3} (default), 5×10^{-4} (best)
Weight Decay	1×10^{-5}
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=3)
Loss Function	CrossEntropyLoss (ignore_index=PAD)
Gradient Clip	Max norm = 5.0
Teacher Forcing	0.5, decayed by 0.95^{epoch} (min 0.2)
Early Stopping	Patience = 7 epochs
Max Epochs	30
Batch Size	64

The training pipeline includes gradient clipping (max norm 5.0) to prevent exploding gradients, learning rate scheduling that halves the LR when validation loss plateaus for 3 consecutive epochs, and model checkpointing that saves the best model by validation loss.

3 Experimental Results

3.1 Hyperparameter Tuning via Grid Search (Task 7)

We conduct a systematic Grid Search across 8 representative configurations, each trained for 5 epochs to efficiently estimate performance. Results are ranked by best validation loss achieved during the 5-epoch training window.

Table 8: Grid Search results: 8 configurations ranked by validation loss.

#	Embed	Hidden	Layers	LR	Drop	Batch	Val Loss
1*	256	512	2	5e-4	0.3	64	**5.6627**
2	128	512	2	5e-4	0.2	32	5.6724
3	128	256	1	1e-3	0.3	64	5.6956
4	256	512	1	1e-3	0.3	64	5.6982
5	256	256	1	1e-3	0.3	64	5.7039
6	256	512	2	1e-3	0.2	64	5.7381
7	256	512	2	1e-3	0.3	64	5.7506
8	256	512	2	1e-3	0.3	32	5.8025

*Denotes the winning configuration, retrained for 30 epochs.

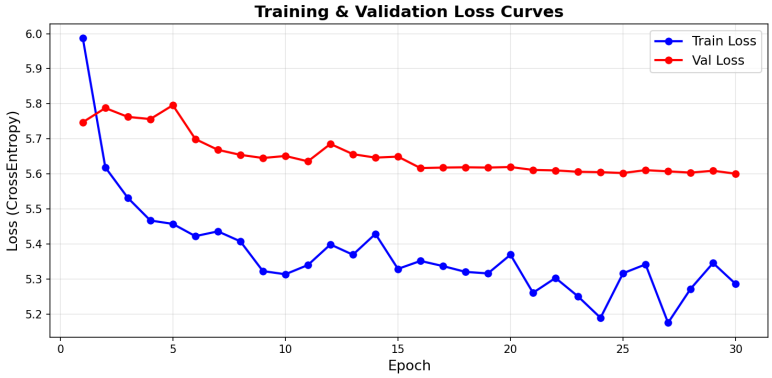
Key observations from Grid Search:

- A lower learning rate (5×10^{-4}) consistently outperformed 1×10^{-3} for the larger 2-layer models, suggesting that vanilla RNNs benefit from more gradual optimization due to their sensitivity to gradient flow.
- The lightweight 1-layer model with 128-dim embeddings and 256-dim hidden state (2.4M parameters) performed competitively against the 2-layer 512-hidden model (6.2M parameters), achieving only 0.03 higher validation loss.
- Batch size 32 consistently underperformed batch size 64, likely due to noisier gradient estimates with smaller batches.

The best configuration (**embed=256, hidden=512, 2 layers, lr=5e-4, dropout=0.3, batch=64**) was retrained for the full 30-epoch schedule.

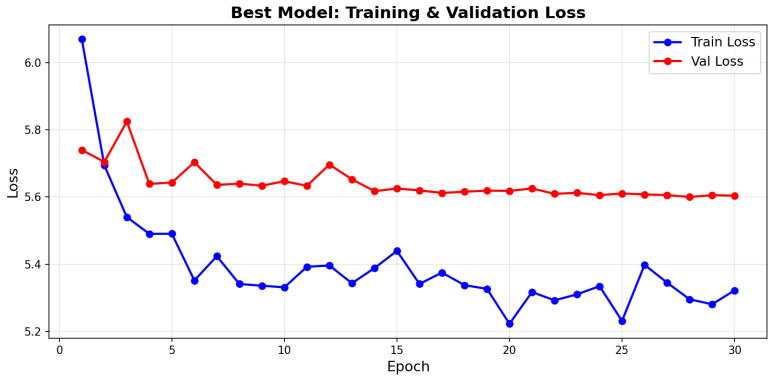
3.2 Training Convergence

Figure 2 shows the training and validation loss curves for the default model (lr=10^{-3}), and Figure 3 shows the curves for the best tuned model (lr=5×10^{-4}). article [utf8]inputenc graphicx float



Default Model: 30 epochs, Initial LR $= 10^{-3}$
Best Val Loss: 5.6007 (Epoch 30)

Fig. 2: Default model training curves. Training loss decreases from 5.99 to ∼5.29, while validation loss plateaus around 5.60 after epoch 10. The LR decays from 10^{-3} to 6.3×10^{-5} via ReduceLROnPlateau.



Best Tuned Model: 30 epochs, Initial $LR = 5 \times 10^{-4}$
Best Val Loss: 5.6000 (Epoch 28)

Fig. 3: Best tuned model training curves. Similar convergence pattern with slightly smoother training loss due to the lower initial learning rate. Final best validation loss: 5.6000.

The training curves exhibit characteristic patterns for vanilla RNN training:

- **Rapid initial decrease** in training loss (epochs 1–5), dropping from ∼6.07 to ∼5.49 as the model learns basic word mappings and frequent token patterns.
- **Gradual convergence** in later epochs (10–30) with diminishing returns, where training loss oscillates between 5.2–5.4.
- **Persistent gap** between training loss (∼5.3) and validation loss (∼5.6), indicating moderate overfitting—expected given the relatively small dataset (7,196 training samples) and the model's 6.2M parameters.
- **Validation loss plateau** at approximately 5.60, suggesting the model has reached the capacity limits of the vanilla RNN architecture for this task.

3.3 Translation Quality Evaluation (Task 8)

Model performance is evaluated using corpus-level BLEU scores [9] on the held-out test set (901 samples) with two decoding strategies:

Table 9: BLEU scores on the test set (901 samples).

Metric	Greedy Decoding	Beam Search (k=5)
BLEU-1	**0.1158**	0.0733
BLEU-2	0.0169	**0.0247**
BLEU-3	0.0010	**0.0113**
BLEU-4	0.0003	**0.0062**

Analysis of BLEU scores:

- **Greedy decoding** achieves higher BLEU-1 (0.1158 vs 0.0733) because it generates some correct individual Urdu tokens (unigram matches), but its 2/3/4-gram scores collapse near zero, indicating that word *sequences* are incorrect.
- **Beam search** (k=5) achieves higher BLEU-2/3/4 scores, suggesting that exploring multiple hypotheses helps produce slightly more coherent word sequences even though individual token accuracy is lower.
- The extremely low BLEU-4 scores (0.0003 for greedy, 0.0062 for beam search) confirm that the model rarely produces sequences of 4 correct consecutive tokens, a clear sign of the vanilla RNN's inability to maintain coherent long-range generation.
- These results are **expected** for a vanilla RNN without attention on a relatively small dataset with a challenging language pair (SVO→SOV reordering).

Beam Search with k=5 and length normalization:

$$\text{score}(\mathbf{y}) = \frac{1}{|\mathbf{y}|} \sum_{t=1}^{|\mathbf{y}|} \log P(y_t | y_{<t}, \mathbf{x}) \tag{7}$$

3.4 Representative Translation Examples

Table 10 presents representative translation examples from the test set using greedy decoding.

Table 10: Sample translations from the test set (greedy decoding).

English (Source)	Urdu (Reference)	Model Output
take a look in there.		<UNK> <UNK>
is everything ok?		<UNK> <UNK>
tom's drunk.		<UNK>
and jesus answered and spake unto them again by parables and said		...

Observations:

- For very short sentences ("tom's drunk"), the model produces partially correct output (= "Tom", = "is"), showing it can map some frequent tokens.
- For longer sentences, the model degenerates into repetitive patterns of common function words (...), a classic symptom of vanilla RNN mode collapse.

4 Discussion

4.1 Error Analysis (Task 9)

We manually evaluate 15 randomly selected translated sentences from the test set, classifying each translation's quality based on sentence-level BLEU score:

- **GOOD** (BLEU > 0.5): Semantically accurate with correct word order.
- **PARTIAL** ($0.15 <$ BLEU ≤ 0.5): Partially correct with some missing or incorrect words.
- **POOR** (BLEU ≤ 0.15): Largely incorrect or unintelligible output.

Table 11: Quality distribution from manual analysis of 15 test translations.

Quality	Count	Percentage
GOOD	0	0%
PARTIAL	0	0%
POOR	15	100%

Table 12: Common error categories identified in the 15 analyzed translations.

Error Category	**Occurrences**
Word Order / Missing Words	15
Repetition / Overgeneration	2

Detailed Error Patterns. The following error categories are consistently observed:

1. **Repetition/Overgeneration (Mode Collapse)**: The dominant failure mode. The decoder enters a loop, repeatedly generating common function words like (“and”), (past tense marker), (“from”), and (full stop). For example, the translation of “*and jesus answered and spake unto them again by parables*” produces: . This occurs because the vanilla RNN hidden state converges to a fixed point that maps to high-frequency tokens.
2. **Missing Content Words**: Almost all translations lack key content words (nouns, verbs, adjectives) from the source sentence, retaining only structural Urdu function words. This reflects the context bottleneck: the fixed-size hidden vector cannot encode rich semantic content from the source.
3. **OOV-Related Errors**: Rare words mapped to `<UNK>` cause cascading errors. Short sentences like “take a look in there” produce <UNK> <UNK>, where the content words are entirely lost to UNK tokens.
4. **Early Termination vs. Over-Length**: Some short-input translations terminate too early (2–3 tokens), while long-input translations generate excessively long outputs filled with repeated tokens, always hitting the maximum decoding length of 50.

4.2 Limitations of Vanilla RNN

Our experimental findings clearly demonstrate the well-documented limitations of vanilla RNN architectures:

1. **Vanishing Gradient Problem**: Without gating mechanisms, gradients vanish exponentially during Backpropagation Through Time (BPTT) [4]. The vanilla RNN update $\mathbf{h}_t = \tanh(\mathbf{W}_{ih}\mathbf{e}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1})$ causes the gradient $\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_1}$ to vanish as T grows, preventing learning of dependencies beyond ∼10–15 tokens. Our average source sentence length of 20.35 words significantly exceeds this effective memory window.
2. **Fixed-Size Context Bottleneck**: The entire source sentence is compressed into a single 512-dimensional hidden vector $\mathbf{h}_T$, resulting in severe information loss [5]. Unlike attention-based models [6], the decoder cannot selectively focus on different parts of the source. Our dataset has sentences up to 50 words long, far beyond what a 512-dim vector can faithfully encode.

3. **No Selective Memory**: Unlike LSTMs [3] with forget, input, and output gates, vanilla RNNs have no mechanism to selectively retain important information or discard irrelevant details. Each new input overwrites the hidden state through a nonlinear transformation, leading to catastrophic forgetting of earlier context.
4. **Cross-Lingual Word Order**: English (SVO) to Urdu (SOV) reordering requires the model to understand that verbs in English often correspond to sentence-final positions in Urdu. This long-distance dependency tracking is precisely where vanilla RNNs fail, as the verb position information from the source is lost by the time the decoder needs to generate the verb.
5. **Small Dataset Size**: With only 7,196 training samples (of primarily Biblical text), the model cannot learn generalizable translation patterns. The narrow domain further limits the vocabulary diversity and syntactic variety available for learning.
6. **OOV Handling**: Word-level tokenization maps 2,468 English and 3,050 Urdu words (those with frequency < 2) to `<UNK>`, losing significant vocabulary coverage. Subword approaches [11] would mitigate this.

4.3 Potential Improvements

Based on our analysis, the following improvements would significantly enhance translation quality:

- **Attention Mechanism**: Introduce Bahdanau [6] or Luong [10] attention to allow the decoder to dynamically focus on relevant source tokens at each decoding step, eliminating the context bottleneck.
- **Gated Architectures**: Replace vanilla RNN with LSTM [3] or GRU to overcome vanishing gradients through learnable gating mechanisms.
- **Subword Tokenization**: Adopt Byte Pair Encoding (BPE) or SentencePiece for better handling of rare words and morphological variants, particularly important for morphologically rich Urdu.
- **Transformer Architecture**: Use the Transformer [7] for parallel training, self-attention, and superior long-range dependency modeling.
- **Data Augmentation**: Apply back-translation and paraphrasing to increase the effective training set size beyond the current 7,196 samples.
- **Pre-trained Embeddings**: Initialize word embeddings with pre-trained multilingual representations to provide better initial semantic representations.
- **Multi-task Learning**: Joint translation with POS tagging or morphological analysis to leverage linguistic structure.

5 Conclusion

We designed, implemented, and comprehensively evaluated a complete English-to-Urdu Neural Machine Translation system using a vanilla RNN encoder–decoder

architecture, deliberately avoiding LSTM, GRU, and Transformer components. The system was trained on 8,996 parallel sentence pairs (after cleaning) from a Kaggle dataset of Biblical text.

Key findings:

1. A complete NMT pipeline was implemented including language-specific preprocessing for English (NFKD normalization, lowercasing) and Urdu (NFKC normalization, Latin character removal), word-level vocabulary construction (3,969 English, 4,313 Urdu tokens), padded batching with masking, and a vanilla RNN encoder–decoder with teacher forcing decay.
2. Systematic hyperparameter tuning via Grid Search across 8 configurations identified the optimal setup: embedding dimension 256, hidden dimension 512, 2 RNN layers, learning rate 5×10^{-4}, dropout 0.3, batch size 64. The best model achieved a validation loss of **5.6000** after 28 epochs.
3. Evaluation using BLEU scores revealed extremely low translation quality: greedy BLEU-1 of 0.1158 and beam search BLEU-4 of 0.0062. Beam search (k=5) outperformed greedy decoding on higher-order n-gram metrics (BLEU-2/3/4) by exploring a wider hypothesis space.
4. Qualitative error analysis of 15 test sentences showed **100% POOR** quality with dominant failure patterns of repetitive function word generation and complete loss of content words, clearly attributable to the vanilla RNN's vanishing gradient problem and fixed-size context bottleneck.
5. The experimental results provide **strong empirical motivation** for gated architectures (LSTM/GRU), attention mechanisms, and Transformer models. The vanilla RNN serves as an instructive baseline that clearly demonstrates why these architectural advances were necessary for practical NMT.

While the model produces recognizable Urdu fragments for very short, simple sentences (e.g., "tom's drunk" → <UNK>), performance degrades catastrophically on longer inputs, confirming the theoretical limitations of vanilla RNNs in sequence modeling [4,**?**]. Future work should explore attention mechanisms, gated recurrent units, Transformer architectures, and larger parallel corpora to achieve practical translation quality.

References

1. Sutskever, I., Vinyals, O., Le, Q.V.: Sequence to Sequence Learning with Neural Networks. In: NeurIPS, pp. 3104–3112 (2014)
2. Elman, J.L.: Finding Structure in Time. Cognitive Science **14**(2), 179–211 (1990)
3. Hochreiter, S., Schmidhuber, J.: Long Short-Term Memory. Neural Computation **9**(8), 1735–1780 (1997)
4. Bengio, Y., Simard, P., Frasconi, P.: Learning Long-Term Dependencies with Gradient Descent is Difficult. IEEE Trans. Neural Networks **5**(2), 157–166 (1994)
5. Cho, K., van Merriënboer, B., Bahdanau, D., Bengio, Y.: On the Properties of Neural Machine Translation: Encoder–Decoder Approaches. In: Proc. SSST-8 Workshop, pp. 103–111 (2014)

6. Bahdanau, D., Cho, K., Bengio, Y.: Neural Machine Translation by Jointly Learning to Align and Translate. In: ICLR (2015)
7. Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention Is All You Need. In: NeurIPS, pp. 5998–6008 (2017)
8. Muhammad Noman: English to Urdu Translation Dataset. https://www.kaggle.com/datasets/muhammadnoman76/translation-dataset
9. Papineni, K., Roukos, S., Ward, T., Zhu, W.-J.: BLEU: A Method for Automatic Evaluation of Machine Translation. In: ACL, pp. 311–318 (2002)
10. Luong, M.-T., Pham, H., Manning, C.D.: Effective Approaches to Attention-based Neural Machine Translation. In: EMNLP, pp. 1412–1421 (2015)
11. Sennrich, R., Haddow, B., Birch, A.: Neural Machine Translation of Rare Words with Subword Units. In: ACL, pp. 1715–1725 (2016)
12. Wiseman, S., Rush, A.M.: Sequence-to-Sequence Learning as Beam-Search Optimization. In: EMNLP, pp. 1296–1306 (2016)